NetCon V2 Full Documentation

A Modular Networking and Lifecycle Management Framework for Roblox
Developed by CTTeddy

https://github.com/CTTeddy/NetConV2/

📌 Overview

NetCon V2 is a modular networking framework designed to structure client-server communication and handle service/controller lifecycles cleanly in Roblox projects. Unlike V1, which relies on a single shared channel, V2 dynamically manages individual remote objects and introduces an automated loading mechanism inspired by frameworks like Knit.

Here is a detailed, side-by-side comparison between **Knit**, **NetCon V1**, and **NetCon V2** in English:

Comparison Table: Knit vs. NetCon V1 vs. NetCon V2

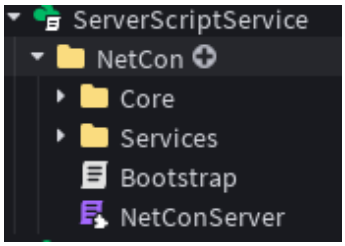
Feature / Aspect	Knit (Roblox Framework)	NetCon V1 (Legacy)	NetCon V2 (Current)
Primary Focus	A comprehensive, full-featured framework for Roblox handling networking (Signals/Remote Properties), services, controllers, and middleware.	A lightweight, single-channel networking wrapper designed to route all actions through one shared remote.	A modular networking and lifecycle framework combining Knit-like service/controller structure with independent remote management.
Network Architecture	Uses dedicated signals (Knit.CreateSignal) for individual events and properties dynamically.	Relies on a single global channel (NetConMainChannel) where all actions are filtered via an actionName parameter.	Dynamically creates and manages separate RemoteEvent and RemoteFunction instances per event/function name.
Service & Controller Loading	Automatically boots up Services (Server) and Controllers (Client) via built-in lifecycle management.	Lacks automated loading; requires manual script creation and explicit registration for each action.	Features automated loaders (ServiceLoader and ControllerLoader) that scan directories and run :Start() methods automatically,

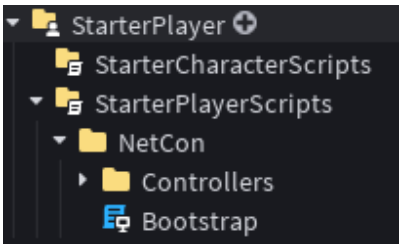
			heavily inspired by Knit
Security & Rate Limiting	Provides middleware support to intercept and validate remote calls globally or per signal.	Includes a fixed, built-in global spam protection limit (SPAM_LIMIT and time window) on the server.	Offers fine-grained, event-specific control via customizable **Validators** and per-player **Cooldowns** directly on each event object.
Scope & Complexity	Large-scale framework suited for complex, production-ready games with extensive cross-service communication.	Extremely simple and minimal, ideal for small scripts or quick setups requiring minimal boilerplate.	Balanced and lightweight; provides structured modular architecture without the heavy boilerplate of larger frameworks.

📂 Project Architecture

NetCon V2 organizes its code into dedicated folders across the workspace hierarchy:

- **ReplicatedStorage / NetCon**
  - Events/ (Auto-generated folder containing RemoteEvent instances)
  - Shared/ (Contains the client module loader and NetCon client API)
- **ServerScriptService / NetCon**
  - Core/ (Event, Network, Registry, and ServiceLoader modules)
  - Services/ (Server-side service modules loaded automatically)
  - Bootstrap – Starts The System Server Side
  - NetConServer (Main server initialization script)
- **StarterPlayer / StarterPlayerScripts / NetCon**
  - Controllers/ (Client-side controller modules loaded automatically)
  - Bootstrap – Starts The System Client Side





⚙️ Server-Side API (NetConServer)

The server script initializes the core network system, loads all services, and manages remote communication.

Methods:

- **NetConServer.Init()**
  - Starts the network layer, loads and runs all server services via ServiceLoader, and logs the startup warning.

Example:

```
local NetConServer = require(script.Parent.NetConServer)
NetConServer.Init()
```

- **NetConServer.Event(name)**
  - Creates or retrieves a managed server event wrapper.
  - **Event Object Methods:**
    - :OnServer(callback): Registers the function executed when a client triggers the event.
    - :SetCooldown(seconds): Applies a rate-limit cooldown per player.
    - :SetValidator(callback): Validates incoming payloads; returns false to block execution.
    - :FireClient(player, data): Sends data to a specific player.
    - :FireAll(data): Broadcasts data to all connected clients.
- **NetConServer.Function(name)**
  - Creates or retrieves a RemoteFunction instance.
- **NetConServer.Register(name, callback)**
  - Quick helper to create an event and assign its server callback simultaneously.
- **NetConServer.GetService(name)**
  - Fetches a running service instance from the central registry.

💻 Client-Side API (NetCon)

The client module handles communication targeting the server-side architecture.

Methods:

- **NetCon.Init()**

    - Initializes the client side and outputs the startup signature.
- **NetCon.Event(name) / NetCon.Fire(name, data)**
    - Safely references or fires an event payload to the server.

Example:

```
local NetCon = require(game.ReplicatedStorage.NetCon.Shared.NetCon)
NetCon.Fire("SampleEvent", { action = "test" })
```

- **NetCon.On(name, callback)**
    - Listens for triggers sent from the server.
- **NetCon.Function(name)**
    - References a remote function.
    - **Function Object Methods:**
        - :Invoke(data): Sends an invocation to the server and yields for a response.

🔄 Service & Controller Lifecycles

NetCon V2 automatically scans specific directories to handle execution flows.

Server Services (ServerScriptService > NetCon > Services)

Any ModuleScript inside this folder is automatically loaded. If it features a :Start() method, it runs inside an asynchronous protected call.

```
local MyService = {}

function MyService:Start()
    print("MyService started successfully.")
end

return MyService
```

Client Controllers (StarterPlayerScripts > NetCon > Controllers)

Any ModuleScript placed here is loaded when the client boots up, executing its :Start() method automatically via task.spawn.

```lua
local MyController = {}

function MyController:Start()
    print("MyController started successfully.")
end

return MyController
```

⚖️ Strict Terms of Use & Copyright Notice

By accessing, downloading, integrating, or utilizing NetCon v2, you agree to the following legally binding terms:

All Rights Reserved: Exclusive intellectual property of the developer.

Absolute Prohibition of Modification: Altering any portion of the core source code is strictly prohibited.

The startup logs and copyright warnings containing **NetCon V2 Server/Client By CTTeddy cannot be removed or altered under any circumstances**.

Strict Anti-Redistribution & Anti-Monetization Policy: Redistribution, resale, or unauthorized distribution is forbidden.

Enforcement & Legal Action: Unauthorized use will result in DMCA takedowns and formal legal action.

👥 Credits & Metadata

Developer: CTTeddy

Project: NetCon v2 Simple Network & Communication System